

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»
ФАКУЛЬТЕТ СИСТЕМ УПРАВЛЕНИЯ И РОБОТОТЕХНИКИ

РАЗРАБОТКА СИМУЛЯЦИОННОЙ СИСТЕМЫ
СЛЕДОВАНИЯ ДИФФЕРЕНЦИАЛЬНОГО РОБОТА
ЗА ЛИДЕРОМ ПО ARUCO-МАРКЕРУ В ROS 2

Аннотация – В работе разработана симуляционная система следования мобильного робота за роботом-лидером. Проект реализован на ROS 2 Jazzy и Gazebo Sim. В сцене одновременно запускаются два дифференциальных робота: лидер движется по заранее заданному профилю скоростей и несет ArUco-маркер ID 42, а робот-последователь обнаруживает этот маркер камерой, оценивает его положение, фильтрует измерения фильтром Калмана и формирует команды скорости с помощью PD-регулятора. Для повышения устойчивости реализован режим памяти траектории: если маркер временно пропадает из поля зрения, последователь продолжает движение по сохраненному пути лидера.

Ключевые слова: ROS 2, Gazebo Sim, differential drive, leader-follower, ArUco, OpenCV, Kalman filter, PD controller, path memory.

Выполнили

Май Хоанг Куан

Ису: 407882,

*ТГ: @quanmh_25,
itmoquanmh@gmail.com*

Проверил

Живицкий Андрей Юрьевич

19 июня 2026 г.

Содержание

1	Введение	2
2	Архитектура системы	2
2.1	Структура проекта	2
2.2	Запуск системы	2
2.3	Роботы и пространства имен	3
2.4	Сенсоры и приводы	3
3	ROS 2 интерфейсы	4
3.1	Ключевые топики	4
3.2	TF-фреймы	4
4	Алгоритмы управления	4
4.1	Движение лидера	4
4.2	Обнаружение ArUco-маркера	5
4.3	Фильтр Калмана	5
4.4	PD-регулятор по ArUco	6
4.5	Ограничение скорости	6
4.6	Память траектории лидера	7
4.7	Следование по сохраненной траектории	7
5	Реализация	8
5.1	Основные файлы	8
5.2	Параметры последователя	8
6	Запуск и тестирование	9
6.1	Запуск через Docker	9
6.2	Нативный запуск	9
6.3	Полезные команды проверки	10
7	Результаты	10
8	Обсуждение	11
9	Заключение	12

1 Введение

Задача следования мобильного робота за лидером встречается в складской робототехнике, групповой навигации, сервисных роботах и автономных транспортных системах. В простейшем варианте один робот выполняет роль лидера, а второй робот должен сохранять безопасную дистанцию, выравниваться по направлению движения и продолжать движение даже при кратковременной потере визуального ориентира.

Цель данного проекта состоит в разработке воспроизводимой симуляционной системы, в которой:

- в Gazebo Sim запускаются два дифференциальных робота;
- лидер движется по заданному сценарию скоростей;
- лидер несет визуальный ArUco-маркер ID 42;
- последователь обнаруживает маркер с помощью камеры и OpenCV;
- положение маркера фильтруется фильтром Калмана;
- команды скорости формируются PD-регулятором;
- при потере маркера используется сохраненная траектория лидера;
- траектории лидера и последователя визуализируются в RViz.

Проект оформлен как ROS 2 workspace и содержит два основных пакета: `follower_scene` и `follower_control`. Для удобства запуска также подготовлены `Dockerfile` и `docker-compose.yaml`.

2 Архитектура системы

2.1 Структура проекта

Проект разделен на пакет симуляционной сцены и пакет управляющих узлов. Такое разделение упрощает отладку: модель робота, мир Gazebo, мосты и контроллеры находятся отдельно от алгоритмов обработки изображения и управления движением.

Пакет	Назначение
<code>follower_scene</code>	Мир Gazebo, Xacro-модели лидера и последователя, конфигурация <code>ros2_control</code> , <code>diff_drive_controller</code> , мост Gazebo–ROS и RViz.
<code>follower_control</code>	Узлы <code>leader_control_node</code> и <code>follower_control_node</code> , детектор ArUco, фильтр Калмана, PD-регулятор и вспомогательные функции.

Таблица 1: Основные пакеты проекта

2.2 Запуск системы

Главный launch-файл проекта:

```
workspace/src/follower_control/launch/run.launch.py
```

Он подключает сцену из `follower_scene/launch/bringup.launch.py`, открывает `rqt_image_view` для просмотра отладочного изображения и через задержку 8 секунд запускает управляющие узлы лидера и последователя. Задержка нужна для того, чтобы Gazebo, мосты и контроллеры успели инициализироваться.

Launch-файл сцены выполняет следующие действия:

- запускает Gazebo Sim с миром `map.sdf`;
- добавляет путь к моделям через `GZ_SIM_RESOURCE_PATH`;
- запускает `ros_gz_bridge` по конфигурации `ros_gz_bridge.yaml`;
- создает робота-последователя в позиции $(0.0, 0.0, 0.0)$;
- создает робота-лидера в позиции $(2.0, 0.0, 0.0)$;
- запускает `robot_state_publisher` для обоих роботов;
- запускает `joint_state_broadcaster` и `diff_drive_controller` для обоих роботов;
- публикует статические TF-преобразования `world -> follower/odom` и `world -> leader/odom`;
- открывает RViz с подготовленной конфигурацией.

2.3 Роботы и пространства имен

В проекте используются два одинаковых по кинематике дифференциальных робота, но с разными ролями:

- `/leader` — робот-лидер, который движется по заданному профилю скоростей;
- `/follower` — робот-последователь, который наблюдает лидера камерой и следует за ним.

Разделение по пространствам имен позволяет запускать два независимых набора контроллеров, TF-фреймов и команд скорости без конфликтов имен. Для каждого робота используется собственный controller manager: `/leader/controller_manager` и `/follower/controller_manager`.

2.4 Сенсоры и приводы

Оба робота управляются через `ros2_control` и `diff_drive_controller`. Основные параметры дифференциальной платформы:

$$R = 0.1 \text{ м}, \quad L = 0.65 \text{ м},$$

где R — радиус колеса, а L — расстояние между левым и правым колесом.

Команды скорости публикуются в формате `geometry_msgs/msg/TwistStamped`. Для лидера используется топик:

`/leader/diff_drive_controller/cmd_vel`

для последователя:

`/follower/diff_drive_controller/cmd_vel`

Робот-последователь оснащен камерой и 2D-лидаром. В текущей реализации основным сенсором алгоритма следования является камера, публикующая изображение в `/follower/camera/image_raw`. Лидар добавлен в модель и проброшен в ROS 2 как `/follower/scan`, что оставляет возможность расширить проект реакцией на препятствия.

3 ROS 2 интерфейсы

3.1 Ключевые топики

Топик	Тип сообщения	Назначение
/clock	roscpp_msgs/msg/Clock	Симуляционное время из Gazebo.
/follower/camera/image_raw	sensor_msgs/msg/Image	Изображение с камеры последователя.
/camera/image_debug	sensor_msgs/msg/Image	Отладочное изображение с нарисованным ArUco-маркером и осями pose.
/follower/scan	sensor_msgs/msg/LaserScan	Данные 2D-лидара последователя.
/leader/diff_drive_controller/cmd_vel	geometry_msgs/msg/TwistStamped	Команды скорости для лидера.
/follower/diff_drive_controller/cmd_vel	geometry_msgs/msg/TwistStamped	Команды скорости для последователя.
/leader/visual_path	nav_msgs/msg/Path	Траектория лидера для отображения в RViz.
/follower/visual_path	nav_msgs/msg/Path	Траектория последователя для отображения в RViz.
/tf, /tf_static	tf2_msgs/msg/TFMessage	Дерево координат роботов, маркера и целевой точки памяти пути.

3.2 TF-фреймы

Для визуализации и вычисления траектории используются следующие важные фреймы:

- world — общий мировой фрейм;
- leader/odom, leader/base_link — одометрия и корпус лидера;
- follower/odom, follower/base_link — одометрия и корпус последователя;
- follower/camera_link — фрейм камеры последователя;
- detected_aruco_42 — оцененное положение ArUco-маркера относительно камеры;
- path_memory_target — целевая точка на сохраненной траектории лидера.

4 Алгоритмы управления

4.1 Движение лидера

Узел `leader_control_node` реализует простой сценарий движения. Он публикует команды `TwistStamped` с периодом 0.1 секунды. Профиль движения состоит из последова-

тельности этапов:

Этап	Длительность, с	v , м/с	ω , рад/с
STOP_START	5	0.0	0.0
PULL_AWAY	6	0.5	0.0
SLOW_LEFT	7	0.25	0.18
STRAIGHT_AFTER_LEFT	5	0.35	0.0
TURN_RIGHT	6	0.3	-0.15
STRAIGHT_FINAL	6	0.35	0.0
STOP_END	5	0.0	0.0

Таблица 3: Профиль движения лидера

Такой профиль позволяет проверить несколько режимов работы последователя: прямое движение, плавный поворот влево, поворот вправо и остановку.

4.2 Обнаружение ArUco-маркера

На роботе-лидере закреплена доска с изображением ArUco-маркера ID 42. Файл текстуры маркера расположен в:

`workspace/src/follower_scene/models/aruco_3/textures/marker_42.png`

Последователь получает изображение с камеры и использует OpenCV ArUco dictionary `DICT_4X4_250`. Детектор выполняет:

1. поиск маркеров на изображении;
2. фильтрацию по ID 42;
3. оценку pose через `estimatePoseSingleMarkers`;
4. преобразование `rvec`, `tvec` в двумерное состояние.

Состояние маркера для управления задается как:

$$z = [x_m \quad z_m \quad \psi_m]^T,$$

где x_m — боковое смещение маркера в камере, z_m — расстояние до маркера вперед, а ψ_m — угол поворота маркера относительно камеры.

4.3 Фильтр Калмана

Измерения ArUco-маркера могут шуметь из-за дискретности изображения, перспективы, частоты камеры и ошибок детекции. Для сглаживания используется фильтр Калмана с состоянием:

$$X = [x \quad z \quad \psi]^T.$$

В реализации используются матрицы:

$$F = I, \quad B = -I, \quad H = I.$$

Шаг предсказания:

$$\hat{X}_k = FX_{k-1} + Bu_k, \quad P_k = FP_{k-1}F^T + Q.$$

Шаг коррекции:

$$y_k = z_k - H\hat{X}_k, \quad S_k = HP_kH^T + R, \quad K_k = P_kH^TS_k^{-1},$$

$$X_k = \hat{X}_k + K_k y_k, \quad P_k = (I - K_k H)P_k.$$

Угол ψ нормализуется в диапазон $[-\pi, \pi]$. Отфильтрованное положение публикуется как TF-фрейм `detected_aruco_42`.

4.4 PD-регулятор по ArUco

Когда маркер виден, последователь использует прямое визуальное управление. Требуемая дистанция до лидера:

$$d_{des} = 0.3 \text{ м.}$$

Ошибка по расстоянию:

$$e_d = z_m - d_{des}.$$

PD-регулятор имеет общий вид:

$$u = K_p e + K_d \frac{e - e_{prev}}{\Delta t}.$$

Для движения по видимому ArUco-маркеру используются коэффициенты:

$$K_p = [0.8, 2.0, 0.0], \quad K_d = [0.05, 0.1, 0.0].$$

Вход регулятора задается как:

$$\begin{bmatrix} -e_d & x_m & \psi_m \end{bmatrix}^T.$$

Результат преобразуется в линейную и угловую скорость:

$$v = \text{clamp}(u_0, 0, v_{max}), \quad \omega = \text{clamp}(u_1, -\omega_{max}, \omega_{max}).$$

Если расстояние до маркера становится меньше или равно 0.3 м, робот останавливается.

4.5 Ограничение скорости

Чтобы движение было плавным, максимальная скорость последователя зависит от расстояния до маркера:

- при $z_m < 1.5$ м используется медленный режим;
- при $1.5 \leq z_m < 2.0$ м скорость плавно увеличивается;
- при $z_m \geq 2.0$ м разрешается максимальная скорость.

Основные параметры:

$$v_{max} = 1.2 \text{ м/с}, \quad \omega_{max} = 2.0 \text{ рад/с},$$

$$v_{slow} = 0.35 \text{ м/с}, \quad v_{cruise} = 0.75 \text{ м/с}.$$

Если маркер виден, но сильно смещен по боковой оси или по yaw, включается recovery-режим. В этом режиме робот не останавливается слишком резко и получает возможность снова выровняться по маркеру.

4.6 Память траектории лидера

Главная особенность проекта — режим `path memory`. Когда маркер виден, последователь переводит TF-фрейм `detected_aruco_42` в мировой фрейм `world` и сохраняет точки траектории лидера:

$$p_i = (x_i, y_i, \theta_i, s_i),$$

где s_i — накопленная длина пути.

Новая точка добавляется, если расстояние от предыдущей точки больше:

$$\Delta s \geq 0.03 \text{ м.}$$

Чтобы память не росла бесконечно, максимальное число точек ограничено:

$$N_{max} = 3000.$$

4.7 Следование по сохраненной траектории

Если маркер временно пропал, но `use_path_memory = True`, последователь не останавливается сразу. Вместо этого он:

1. находит свое положение `follower/camera_link` в мировом фрейме;
2. ищет ближайшую точку на сохраненной траектории лидера;
3. выбирает целевую точку впереди по траектории;
4. интерполирует положение цели по длине пути;
5. публикует цель как TF-фрейм `path_memory_target`;
6. управляет движением к этой точке отдельным PD-регулятором.

Целевая длина пути выбирается как:

$$s_{target} = \min(s_{nearest} + 0.35, s_{allowed}),$$

где

$$s_{allowed} = \max(0, s_{total} - d_{des}).$$

Для управления по памяти пути используются коэффициенты:

$$K_p = [0.8, 1.8, 0.0], \quad K_d = [0.01, 0.01, 0.0].$$

Если робот дошел до последней разрешенной точки и расстояние до цели меньше `path_goal_tolerance = 0.15` м, последователь останавливается.

5 Реализация

5.1 Основные файлы

Файл	Назначение
<code>follower_control_node.py</code>	Основной узел последователя: подписка на камеру, детекция ArUco, Kalman, PD, path memory, публикация <code>cmd_vel</code> и траекторий.
<code>leader_control_node.py</code>	Узел лидера с заранее заданным профилем линейной и угловой скорости.
<code>modules/aruco_detector.py</code>	Обнаружение ArUco-маркеров и оценка pose по изображению.
<code>modules/kalman_filter.py</code>	Фильтр Калмана для состояния $[x, z, yaw]$.
<code>modules/pd_controller.py</code>	Универсальный PD-регулятор для визуального следования и следования по памяти пути.
<code>follower_robot.xacro</code>	Модель последователя: база, колеса, камера, лидар, <code>ros2_control</code> .
<code>leader_robot.xacro</code>	Модель лидера с ArUco-доской ID 42.
<code>diff_leader.yaml</code> , <code>diff_follower.yaml</code>	Параметры <code>controller manager</code> и <code>diff_drive_controller</code> .
<code>ros_gz_bridge.yaml</code>	Мост Gazebo–ROS для <code>/clock</code> , изображения камеры и лидара.
<code>run.launch.py</code>	Главный запуск полной системы.
<code>bringup.launch.py</code>	Запуск Gazebo, RViz, роботов, TF, bridge и контроллеров.

Таблица 4: Ключевые файлы проекта

5.2 Параметры последователя

В узле `follower_control_node.py` заданы основные параметры поведения:

Параметр	Значение	Смысл
marker_id	42	ID ArUco-маркера на лидере.
marker_size	0.1	Размер маркера, м.
desired_distance	0.3	Желаемая дистанция до лидера, м.
path_goal_tolerance	0.15	Допуск достижения цели памяти пути, м.
max_linear_speed	1.2	Максимальная линейная скорость последователя, м/с.
max_angular_speed	2.0	Максимальная угловая скорость, рад/с.
use_path_memory	True	Разрешает движение по сохраненной траектории при потере маркера.
use_kalman_filter	True	Включает фильтрацию pose маркера.

Таблица 5: Основные параметры последователя

6 Запуск и тестирование

6.1 Запуск через Docker

Из корня репозитория:

```
1 cd /home/quanmh25/quanmh/diffdrive_aruco_follow
2 docker compose build
```

Листинг 1: Сборка Docker-образа

Разрешение контейнеру открывать GUI-окна Gazebo, RViz и rqt_image_view:

```
1 xhost +local:docker
```

Листинг 2: Разрешение X11-доступа

Запуск shell внутри контейнера:

```
1 docker compose run --rm ros
```

Листинг 3: Вход в ROS-контейнер

Внутри контейнера:

```
1 colcon build
2 source install/setup.bash
3 ros2 launch follower_control run.launch.py
```

Листинг 4: Сборка и запуск проекта

В этом режиме `docker compose run -rm ros` создает временный контейнер, который удаляется после выхода из shell.

6.2 Нативный запуск

Если ROS 2 Jazzy и зависимости установлены на хосте, проект можно запустить без Docker:

```

1 cd /home/quanmh25/quanmh/diffdrive_aruco_follow/workspace
2 source /opt/ros/jazzy/setup.bash
3 colcon build
4 source install/setup.bash
5 ros2 launch follower_control run.launch.py

```

Листинг 5: Нативная сборка и запуск

Не нужно отдельно запускать `follower_scene bringup.launch.py`, если уже используется `follower_control run.launch.py`. Главный launch-файл уже включает сцену, поэтому двойной запуск может создать дублирование Gazebo, /clock, мостов или контроллеров.

6.3 Полезные команды проверки

Проверка контроллеров:

```

1 ros2 control list_controllers -c /leader/controller_manager
2 ros2 control list_controllers -c /follower/controller_manager

```

Листинг 6: Проверка controller manager

Ожидаемый результат:

```

1 joint_state_broadcaster active
2 diff_drive_controller active

```

Листинг 7: Ожидаемые активные контроллеры

Проверка камеры, отладочного изображения и команд скорости:

```

1 ros2 topic echo /follower/camera/image_raw --once
2 ros2 topic echo /camera/image_debug --once
3 ros2 topic echo /leader/diff_drive_controller/cmd_vel --once
4 ros2 topic echo /follower/diff_drive_controller/cmd_vel --once

```

Листинг 8: Проверка основных топиков

Проверка TF-дерева:

```

1 ros2 run tf2_tools view_frames

```

Листинг 9: Построение TF-дерева

В TF-дереве должны присутствовать связи `leader/odom -> leader/base_link` и `follower/odom -> follower/base_link`. Также при обнаружении маркера должен появляться фрейм `detected_aruco_42`.

7 Результаты

В результате был создан полный ROS 2 pipeline для симуляции следования робота за лидером. Система объединяет Gazebo Sim, две модели дифференциальных роботов, `ros2_control`, OpenCV ArUco, фильтр Калмана, PD-регулятор, сохранение траектории и визуализацию в RViz.

Задача	Результат	Комментарий
Запуск мира Gazebo и двух роботов	Выполнено	Лидер и последователь запускаются в отдельных namespace.
Управление дифференциальными роботами	Выполнено	Используются <code>ros2_control</code> и <code>diff_drive_controller</code> .
Движение лидера по сценарию	Выполнено	Узел лидера публикует заранее заданный профиль <code>TwistStamped</code> .
Получение изображения камеры	Выполнено	Камера последователя публикует <code>/follower/camera/image_raw</code> .
Обнаружение ArUco ID 42	Выполнено	Используется OpenCV ArUco dictionary <code>DICT_4X4_250</code> .
Фильтрация pose маркера	Выполнено	Реализован фильтр Калмана для состояния <code>[x, z, yaw]</code> .
Следование за видимым маркером	Выполнено	Команды формируются PD-регулятором.
Движение при потере маркера	Выполнено	Используется память траектории лидера.
Визуализация траекторий	Выполнено	Публикуются <code>/leader/visual_path</code> и <code>/follower/visual_path</code> .

Таблица 6: Итоги реализации

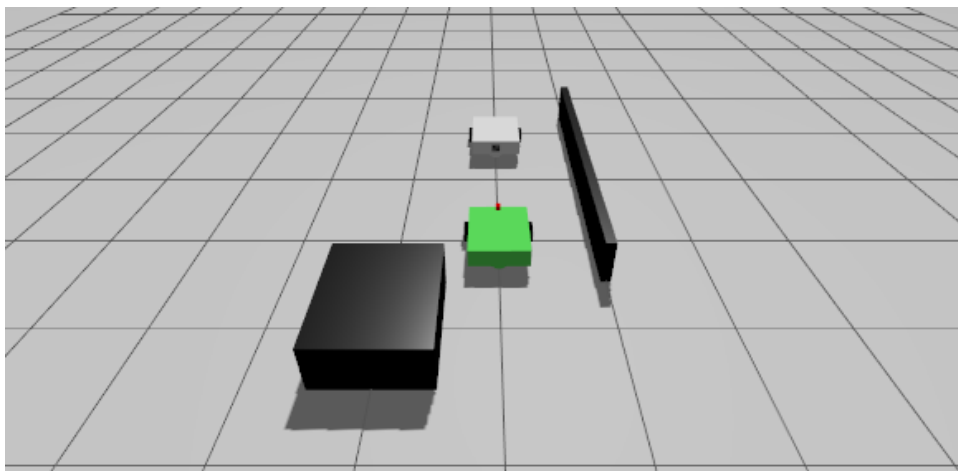


Рис. 1: Демонстрация leader-follower проекта в симуляции

8 Обсуждение

Главное преимущество текущей реализации — простая и наглядная архитектура. Лидер не требует сложного планировщика: он движется по фиксированному сценарию, а последователь решает задачу визуального слежения. Благодаря этому можно отдельно анализировать качество обнаружения маркера, устойчивость фильтра Калмана и поведение PD-регулятора.

Режим памяти траектории делает систему устойчивее к кратковременной потере маркера. Это важно, потому что в реальных условиях ArUco может временно исчезать из поля зрения из-за поворота, перекрытия, размытия изображения или большой дистанции. В проекте сохраненная траектория лидера используется как резервный источник цели.

Ограничения текущей реализации:

- алгоритм зависит от видимости ArUco-маркера и параметров камеры;
- профиль движения лидера задан вручную, без планировщика;
- параметры PD-регуляторов подобраны эмпирически;
- lidar пока проброшен в ROS 2, но не используется в основном алгоритме избегания препятствий;
- система проверена в симуляции, а для реального робота потребуется калибровка камеры и настройка шумов фильтра.

Возможные улучшения:

- добавить автоматическую настройку коэффициентов PD-регуляторов;
- использовать PID, Pure Pursuit или MPC для движения по сохраненной траектории;
- подключить lidar для избегания препятствий;
- добавить оценку качества детекции и отбрасывание выбросов ArUco;
- заменить фиксированный профиль лидера на управление по маршруту или Nav2;
- провести эксперименты с разными дистанциями, скоростями и углами обзора камеры.

9 Заключение

В рамках проекта была разработана симуляционная система следования дифференциального робота за лидером в ROS 2 и Gazebo Sim. Лидер движется по заданной траектории и несет ArUco-маркер ID 42, а последователь обнаруживает маркер камерой, оценивает его pose, фильтрует измерения и формирует команды скорости.

Основной результат работы — готовый leader-follower pipeline, который объединяет Gazebo Sim, `ros2_control`, `diff_drive_controller`, OpenCV ArUco, фильтр Калмана, PD-регулятор, TF и RViz-визуализацию. Дополнительная функция памяти траектории позволяет последователю продолжать движение при временной потере маркера, что делает поведение системы более устойчивым и приближенным к практическим задачам мобильной робототехники.

Список литературы

- [1] ROS 2 Documentation. <https://docs.ros.org/>
- [2] Gazebo Sim Documentation. <https://gazebo-sim.org/docs>
- [3] OpenCV ArUco module. <https://docs.opencv.org/>
- [4] `ros2_control` Documentation. <https://control.ros.org/>